
PRUEBA JENKINS

Aprendiz:

Natalia Chala Chala

Instructor:

Carlos Julio Cadena Sarasty

SENA

Servicio Nacional de Aprendizaje

2026

EVIDENCIA DE APRENDIZAJE

Simulación de Flujo CI/CD con Jenkins, Git y Automatización de Despliegues

Aprendiz:

Natalia Chala Chala

Formación:

Integración Continua y Automatización con Jenkins

Introducción

En esta actividad se realizó una simulación práctica de un flujo completo de integración continua utilizando Jenkins, enfocada en la gestión de cambios, control de versiones y automatización del ciclo de despliegue.

En este ejercicio se simuló un escenario más cercano a un entorno real de desarrollo, incluyendo conflictos de código, correcciones iterativas, validación de pipeline y despliegues progresivos.

2. Objetivos

Objetivo General

Simular un entorno de integración continua con control de errores, múltiples cambios y validación automatizada del pipeline.

Objetivos Específicos

Simular cambios progresivos en un proyecto.

Generar errores controlados para análisis del pipeline.

Validar comportamiento de Jenkins ante fallos.

Ejecutar correcciones y reintentos de build.

Integrar flujo de trabajo con ramas y merges.

3. Escenario de Simulación

Se parte de un proyecto en GitHub con estructura básica y pipeline configurado en Jenkins.

Se definió el siguiente escenario:

Rama principal: main

Rama de desarrollo: dev

Ramas de simulación: feature/*

El objetivo fue simular un equipo de desarrollo con múltiples cambios simultáneos.

Inicio de evidencias

Paso 1 - Crear el contenedor de Jenkins en Docker

1. Primero, crear el volumen persistente
2. Luego Ejecutar Jenkins
3. Verificar que está corriendo

```

PS C:\Users\Aprendiz> docker volume create jenkins_home
jenkins_home
PS C:\Users\Aprendiz> docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home -v //var/run/docker.sock:/var/run/d
ocker.sock jenkins/jenkins:lts
Unable to find image 'jenkins/jenkins:lts' locally
lts: Pulling from jenkins/jenkins
307f8152a55e: Pull complete
9014a3401c36: Pull complete
c2314d42cca1: Pull complete
5834ae4b8774: Pull complete
3bdbd616c406: Pull complete
6b35bb76b8c8: Pull complete
cebc1e77ceba: Pull complete
801814f62c99: Pull complete
4c140080734e: Pull complete
dd00b4ea1f64: Pull complete
76a355a69c2e: Pull complete
ae218d1cfa6e: Pull complete
Digest: sha256:01c992ffef29dcf41c7164e8c16285c657c2368c4943dda8d68c93df54447d5
Status: Downloaded newer image for jenkins/jenkins:lts
3cbb49ef28146551f474972b474816ba1835be17f8064f74ac2c633d4d80ea81
PS C:\Users\Aprendiz> docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
3cbb49ef2814	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	53 seconds ago	Up 48 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp

4. Luego abrir en el navegador

<http://localhost:8080/>

5. Para obtener la contraseña se ejecuta este comando o se visualiza en el contenedor de Docker

```

PS C:\Users\Aprendiz> docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
6ac6a59c9af941808eb087bbd9e11160
PS C:\Users\Aprendiz> |

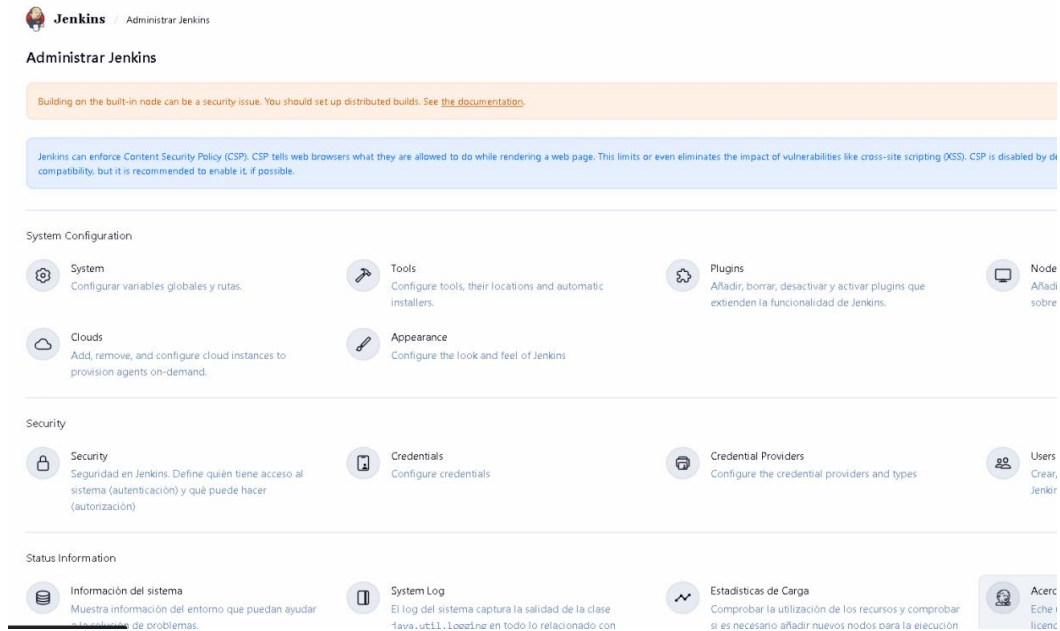
```

Paso 2 – Configuración inicial de Jenkins

1. Haz clic en “Install suggested plugins”
2. Se llenan los campos (usuario, contraseña, nombre completo, e-mail)
3. Se hace click en save

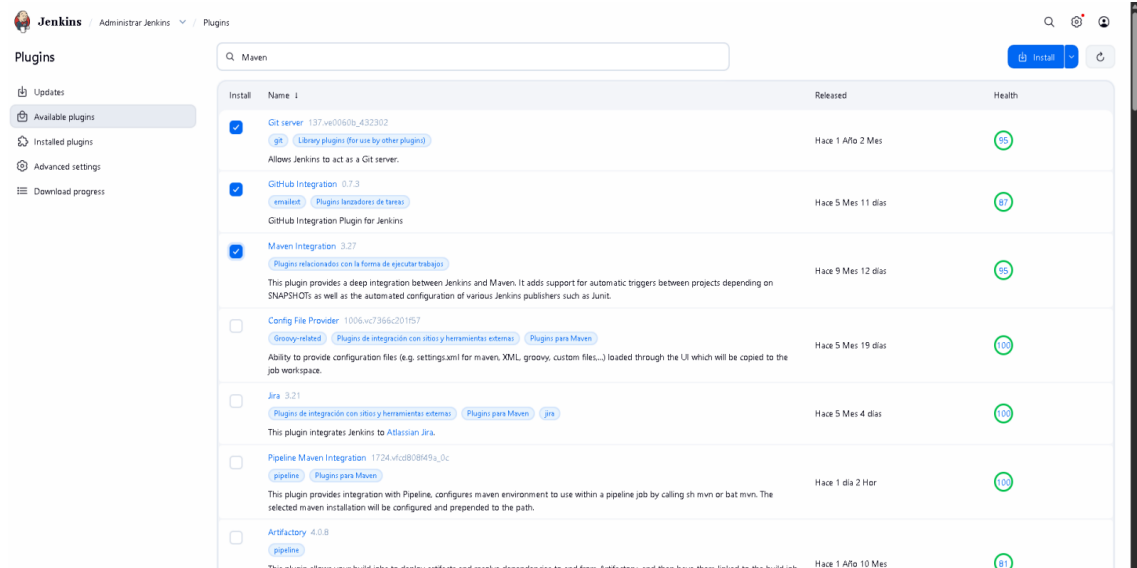
Paso 3 – Instalar plugins adicionales

1. En el panel principal dirigirse a configuración -> plugins -> Available plugins



2. En el buscador, busca estos plugins

- Git server
- Github Integration
- Maven Integration
- Pipeline
- Mail



3. Cuando tenga todo marcado se da en install

Paso 4 – Instalar maven dentro del contenedor Jenkins

1. Entrar al contenedor como root
2. Actualizar paquetes
3. Instalar maven
4. Verificar que se instaló bien
5. Salir del contenedor

```
PS C:\Users\Aprendiz> docker exec -u 0 -it jenkins bash
root@3cbb49ef2814:/# apt update
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [165 kB]
Fetched 10.1 MB in 42s (239 kB/s)
20 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@3cbb49ef2814:/# apt install maven -y
Installing:
  maven

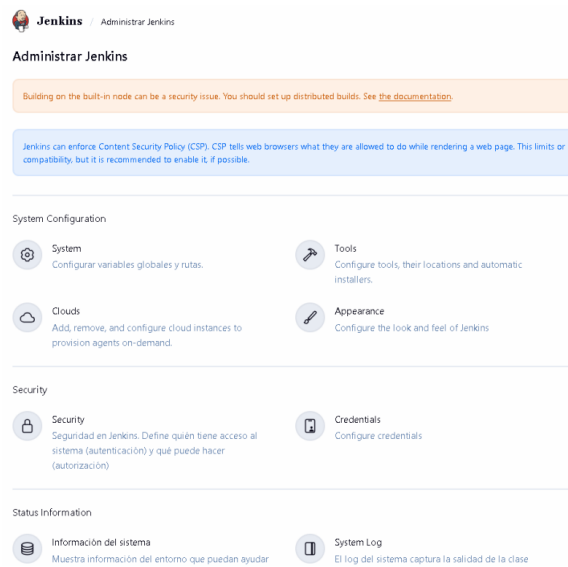
Installing dependencies:
  alsa-topology-conf
  alsa-ucm-conf
  ca-certificates-java
  dbus
  dbus-bin
  dbus-daemon
  dbus-session-bus-common
  dbus-system-bus-common
  default-jre-headless
  java-common
  libaopalliance-java
  libapache-pom-java
  libapparmor1
  libasound2-data
  libasound2t64
  libatinject-jsr330-api-java
  libatomic1
  libavahi-client3
  libavahi-common-data
  libavahi-common3
```

Salida con exit

```
Setting up default-jre-headless (2:1.21-76) ...
Setting up maven (3.9.9-1) ...
update-alternatives: using /usr/share/maven/bin/mvn to provide /usr/bin/mvn (mvn) in auto mode
root@3cbb49ef2814:/# mvn -version
Apache Maven 3.9.9
Maven home: /usr/share/maven
Java version: 21.0.11, vendor: Eclipse Adoptium, runtime: /opt/java/openjdk
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.6.87.2-microsoft-standard-wsl2", arch: "amd64", family: "unix"
root@3cbb49ef2814:/# exit
exit
PS C:\Users\Aprendiz> |
```

Paso 5 – Configurar maven en Jenkins

1. Dirígete a tools



2. Baja hasta encontrar la sección “Maven installations” y haz click en “Add maven”
3. Llena los campos así

instalaciones de Maven

+ Añadir Maven

Maven

Nombre

Maven

MAVEN_HOME

/usr/share/maven

☐ Instalar automáticamente ?

+ Añadir Maven

Save Apply

4. Click en save

Paso 6 – Repositorio GitHub

1. Se puede usar un repositorio que ya tenga, en este caso yo use uno con Spring Boot

<https://github.com/nataliachala1/prueba-jenkins>

Paso 7 – Estructura

1. Se clona el repositorio
`git clone`
2. Se crean las ramas
`git switch -c dev`
`git switch -c feature`
3. Tener todo el trabajo en dev

Paso 8 – Crear el job en jenkins

1. Haz click en “New items”
2. Llena el nombre “prueba-jenkins”
3. Escoge pipeline
4. Click en ok
5. En la configuración de job, baja hasta pipeline y en definition pega este script

```
pipeline {  
  agent any  
  
  stages {  
  
    stage('Clonar repositorio') {  
      steps {  
        git branch: 'dev',  
          url: ' https://github.com/nataliachala1/prueba-jenkins '  
      }  
    }  
  
    stage('Compilar') {  
      steps {  
        sh 'mvn clean compile'  
      }  
    }  
  }  
}
```

```

    }

    stage('Pruebas') {
        steps {
            sh 'mvn test'
        }
    }
}
}
}

```

6. Ejecuta el primer “Build now”

The screenshot shows the Jenkins web interface for a pipeline named 'prueba-jenkins'. The top navigation bar includes 'Status', 'Changes', 'Construir ahora' (Build now), 'Configurar' (Configure), 'Borrar Pipeline' (Delete Pipeline), 'Stages', 'Rename', and 'Pipeline Syntax'. The 'Construir ahora' button is highlighted. Below the navigation bar, there is a section titled 'Enlaces permanentes' (Permanent links) with a list of links to previous builds. The 'Builds' section shows a list of builds with a filter input and a dropdown menu. The builds are listed as follows:

Build Number	Status	Time
#4	Success	14:14
#3	Success	14:12
#2	Success	12:53
#1	Failure	12:47

Paso 9 – Configurar notificaciones por correo

1. Ve a system

The screenshot shows the 'Administrar Jenkins' (Administer Jenkins) page. It includes a warning message about building on the built-in node and a message about Content Security Policy (CSP). The 'System Configuration' section is visible, showing four configuration options:

- System**: Configurar variables globales y rutas.
- Tools**: Configure tools, t installers.
- Clouds**: Add, remove, and configure cloud instances to provision agents on-demand.
- Appearance**: Configure the loc

2. Baja hasta la sección “Notificación por correo electrónico”

- Click en avanzado
- Use SMTP Authentication
- Usa una contraseña de aplicación
- Usar seguridad SSL
- Puerto 465

Notificación por correo electrónico

Servidor de correo saliente (SMTP)

smtp.gmail.com

Sufijo de email por defecto

Avanzado

Use SMTP Authentication

Nombre de usuario

chris006@gmail.com

For security when using authentication it is recommended to enable either TLS or SSL

Contraseña

Usar seguridad SSL

Usar seguridad TLS (STARTTLS)

Puerto de SMTP

465

Dirección para la respuesta

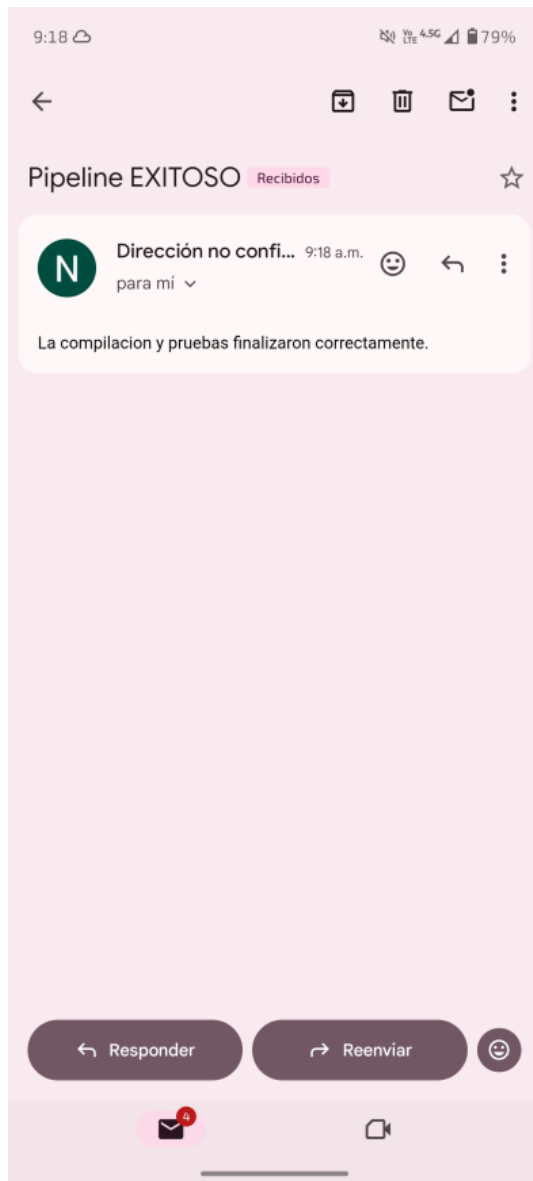
Paso 10 – Actualizar pipeline con Notificaciones

1. Actualizar el script

```
pipeline {  
    agent any  
    stages {  
        stage('Clonar repositorio') {  
            steps {  
                git branch: 'dev',  
                url: 'https://github.com/nataliachala1/prueba-jenkis.git'  
            }  
        }  
        stage('Compilar') {  
            steps {  
                sh 'mvn clean compile'  
            }  
        }  
        stage('Pruebas') {  
            steps {  
                sh 'mvn test'  
            }  
        }  
    }  
}
```

```
    }  
  }  
}  
post {  
  success {  
    mail to: 'chnati006@gmail.com',  
    subject: 'Pipeline EXITOSO',  
    body: 'La compilación y pruebas finalizaron correctamente.'  
  }  
  failure {  
    mail to: 'chnati006@gmail.com',  
    subject: 'Pipeline FALLÓ',  
    body: 'La compilación o pruebas presentaron errores.'  
  }  
}  
}
```

2. Click en “Save”
3. Ejecuta “Build now”
4. Revisa el correo



Paso 11 – Simular las iteraciones de trabajo

Iteración 1

1. Crear la rama feature/login-ui
2. Haz un cambio en el código
git add .
git commit -m "feature: modificacion interfaz login"
git push origin feature/login-ui
3. Luego “Build now”
4. Verificar SUCCES

5. Verificar correo

Iteración 2 Generar error intencional

1. Crea la rama feature/auth-error
2. Crea un error en el código
3. Guarda y sube los cambios
4. Haz un “Build now”
5. Verificar FAILURE

```
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git switch dev
Switched to branch 'dev'
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git switch -c feature/auth-error
Switched to a new branch 'feature/auth-error'
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git add .
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git commit -m "error: fallo intencional"
[feature/auth-error d8c25de] error: fallo intencional
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git push origin feature/auth-error
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 28 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 598 bytes | 598.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 0% (0/remote: Resolving deltas: 50% (1/remote: Resolving deltas: 100% (2/2)
remote:
remote: Create a pull request for 'feature/auth-error' on GitHub by visiting:
remote:   https://github.com/nataliachala1/prueba-jenkins/pull/new/feature/auth-error
remote:
To https://github.com/nataliachala1/prueba-jenkins.git
 * [new branch]      feature/auth-error -> feature/auth-error
PS C:\Users\Aprendiz\Desktop\prueba-jenkins>
```

Iteración 3 Corregir el error

1. Corrige el código que modificaste
2. Guarda
git add .
git commit -m "fix: correccion error en autenticacion"
git push origin feature/auth-error
3. Haz un “Build now”
4. Verifica correo

Iteración 4 Conflicto del merge

1. Haz un merge de feature/login-ui
 - git merge feature/login-ui
2. Ahora intenta hacer merge de feature/auth-error
 - git merge feature/auth-error

```

PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git switch dev
Switched to branch 'dev'
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git merge feature/login-ui
CONFLICT (modify/delete): README.md deleted in HEAD and modified in feature/login-ui. Ve
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git merge feature/auth-error
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
PS C:\Users\Aprendiz\Desktop\prueba-jenkins>

```

3. Resuelve el conflicto

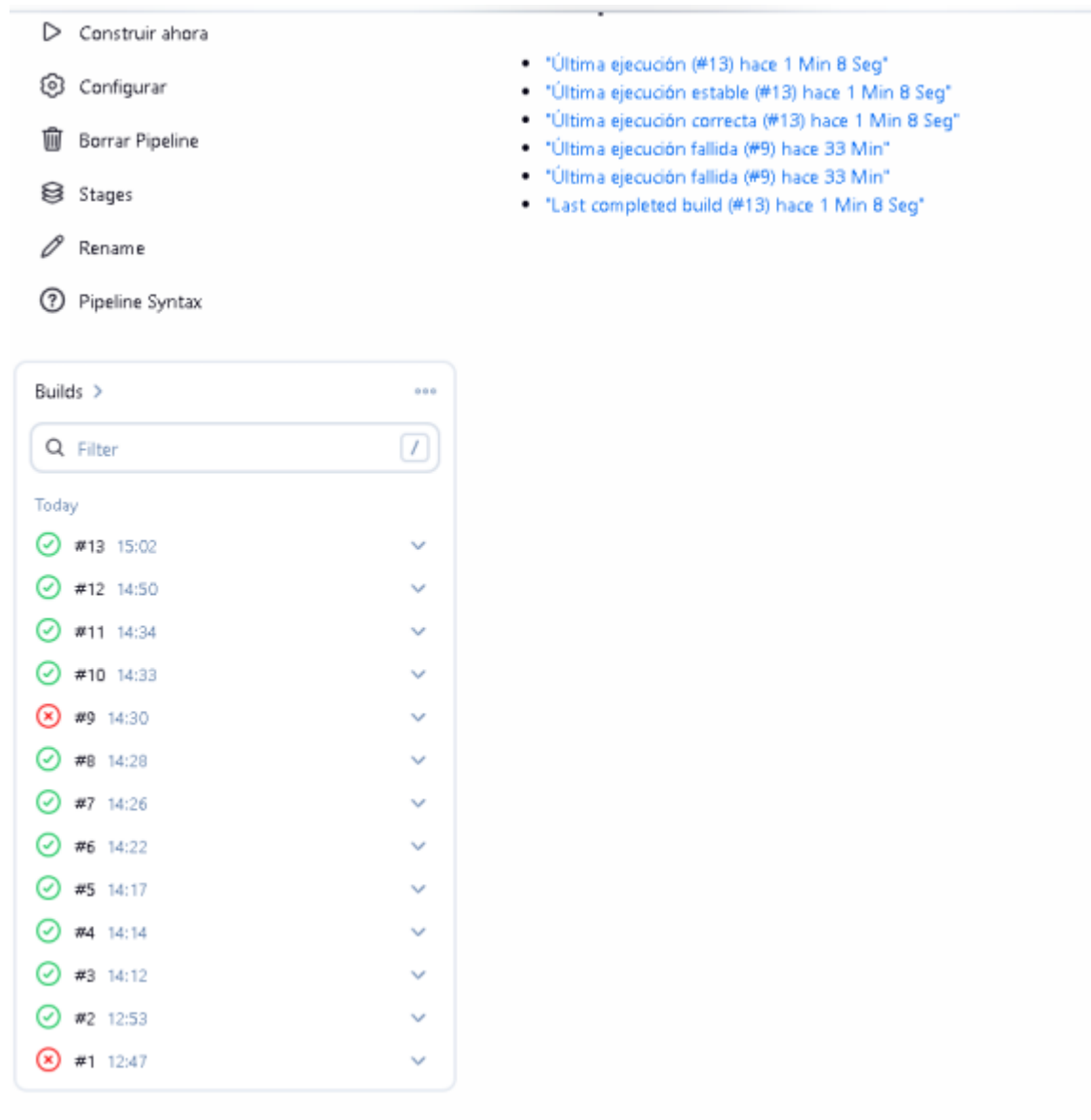
```

PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git rm README.md
rm 'README.md'
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git commit -m "fix: resolucion conflicto merge feature/login-ui y feature/auth-error"
[dev f0a4380] fix: resolucion conflicto merge feature/login-ui y feature/auth-error
PS C:\Users\Aprendiz\Desktop\prueba-jenkins> git push origin dev
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 248 bytes | 248.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nataliachala1/prueba-jenkins.git
  7e776e8..f0a4380 dev -> dev
PS C:\Users\Aprendiz\Desktop\prueba-jenkins>

```

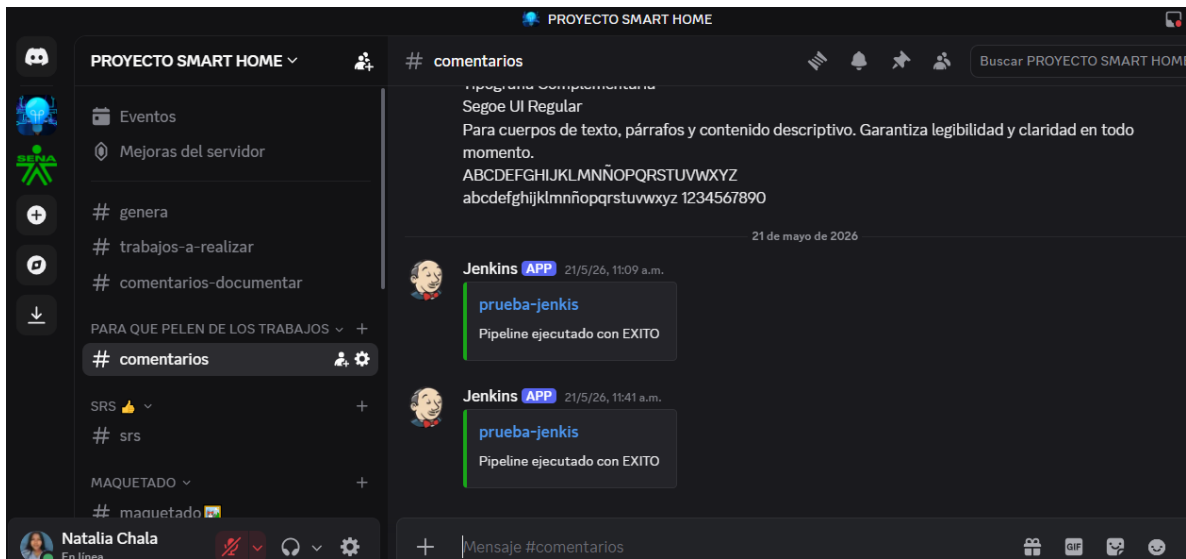
4. Sube los cambios
5. Ve a job, verifica que el pipeline este en la rama “dev”
6. Ejecuta “Build now”

```
[1;34mINFO[1;32mBUILD SUCCESS[1;34mINFO[1m-----  
-----[1m  
[1;34mINFO[1m Total time: 1.651 s  
[1;34mINFO[1m Finished at: 2026-05-21T15:03:05Z  
[1;34mINFO[1m [1m-----  
-----[1m  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Declarative: Post Actions)  
[Pipeline] mail  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```

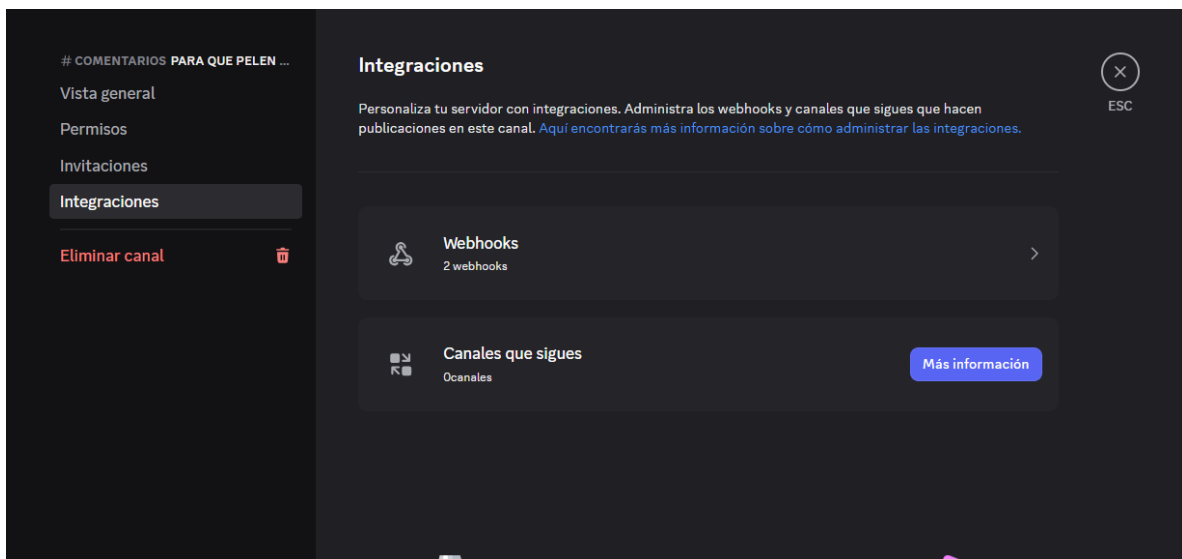


Notificaciones de Discord

1. Entra en discord y selecciona un canal, es importante que sea un canal de texto, en este caso yo seleccione el canal de comentarios



2. Dar en boton de confiuración y luego en editar canal, luego en integraciones, luego en Webhooks



3. Crea un nuevo Webhooks

COMENTARIOS PARA QUE PELEN ...

Vista general

Permisos

Invitaciones

Integraciones

Eliminar canal

Integraciones > Webhooks

ESC

Los webhooks funcionan como una manera fácil de publicar mensajes desde otras aplicaciones y sitios web en Discord mediante la magia del internet. [Aquí encontrarás más información](#). También puedes intentar [crear uno por tu cuenta](#).

Nuevo webhook

Publicando en #comentarios

Captain Hook

Creada el 21 may 2026 por nataliachala1

Captain Hook

Creada el 21 may 2026 por nataliachala1

7:43

🕒 🔕 No WiFi 📶 🔋 18%



Editar webhook



Te recomendamos una imagen de al menos 256 x 256

Nombre

Captain Hook

Canal

comentarios



URL del webhook

<https://discord.com/api/webhooks/1507044110404423682/vXscOrTd3vMrFEe66nByZvxiF1fko2wEhVPAg1lFgnZgvgk95zQkMUxjvBTwVXDwG200>

Copiar

Eliminar

5. En jenkins instala “Discord Notifier” y luego reinicia jenkins
6. En el pipeline agrega esto en post

```
post {  
    success {  
        discordSend description: 'Pipeline ejecutado con EXITO',  
        link: env.BUILD_URL,  
        result: currentBuild.currentResult,  
        title: JOB_NAME,  
        webhookURL: '  
https://discord.com/api/webhooks/1507044110404423682/vXsc0rTd3vMrFEe6  
6nByZvxiF1fko2wEhVPAG1IFgnZgvgk95zQkMUxjvBTwVXDwG200'  
    }  
    failure {  
        discordSend description: 'Pipeline FALLO',  
        link: env.BUILD_URL,  
        result: currentBuild.currentResult,  
        title: JOB_NAME,  
        webhookURL: '  
https://discord.com/api/webhooks/1507044110404423682/vXsc0rTd3vMrFEe6  
6nByZvxiF1fko2wEhVPAG1IFgnZgvgk95zQkMUxjvBTwVXDwG200'  
    }  
}
```

7. Click en save y luego Build now
8. Verifica el mensaj

7:44

🕒 🔕 📶 🔋 18%

← # **comentarios** >
● 1 en línea



Blanco

#FFFFFF

Espacios limpios

5. Tipografía

Tipografía Principal

Segoe UI Bold

Utilizada para titulares, logotipo y elementos destacados. Transmite modernidad y profesionalismo.

ABCDEFGHIJKLMNOPQRSTUVWXYZ

abcdefghijklmnopqrstuvwxyz 1234567890

Tipografía Complementaria

Segoe UI Regular

Para cuerpos de texto, párrafos y contenido descriptivo. Garantiza legibilidad y claridad en todo momento.

ABCDEFGHIJKLMNOPQRSTUVWXYZ

abcdefghijklmnopqrstuvwxyz 1234567890

21 de mayo de 2026



Jenkins APP 21/05/2026 11:09

prueba-jenkins

Pipeline ejecutado con EXITO



Jenkins APP 21/05/2026 11:41

prueba-jenkins

Pipeline ejecutado con EXITO



Mensaje #comentarios



Notificaciones telegram

1. Abre telegram y busca @BotFather
2. Escribe /newbot
3. Colócale un nombre al bot
4. Colocale un user.name

7:48

🔔 📶 16%



BotFather ✓

8,880,143 usuarios al mes



Alright, a new bot. How are we going to call it?
Please choose a name for your bot.

11:22 AM

Jenkins Notifier 11:23 AM ✓

Good. Now let's choose a username for your bot. It must end in `bot`. Like this, for example: TetrisBot or tetris_bot.

11:23 AM

jenkins_notifier_bot 11:23 AM ✓

Sorry, this username is already taken. Please try something different.

11:23 AM

jenkins_notifier_bot_natilla39C 11:26 AM ✓

Sorry, the username must end in `bot`. E.g. Tetris_bot or Tetrisbot.

11:26 AM

jenkins_notifier_natilla_bot 11:27 AM ✓

Done! Congratulations on your new bot. You will find it at t.me/jenkins_notifier_natilla_bot. You can now add a description, about section and profile picture for your bot, see [/help](#) for a list of commands. By the way, when you've finished creating your cool bot, ping our Bot Support if you want a better username for it. Just make sure the bot is fully operational before you do this.

Use this token to access the HTTP API:
8822492274:AAFxUYQbl4CRE68r9q2xo27B7DVhXT7JRpw
Keep your token **secure** and **store it safely**, it can be used by anyone to control your bot.

For a description of the Bot API, see this page:

<https://core.telegram.org/bots/api>

11:27 AM



Open



Mensaje



5. Copia el token
6. Busca tu bot en telegram y escribe un mensaje cualquiera

7:50

🔒 5G LTE 📶 🔋 16%



Jenkins Notifier

bot



📷 Establecer foto del perfil

21 de mayo

/start 11:28 AM ✓✓

Hola 11:28 AM ✓✓

Hola 11:35 AM ✓✓

✓ Pipeline EXITOSO: prueba-jenkis Build #15
finalizo correctamente.

11:41 AM



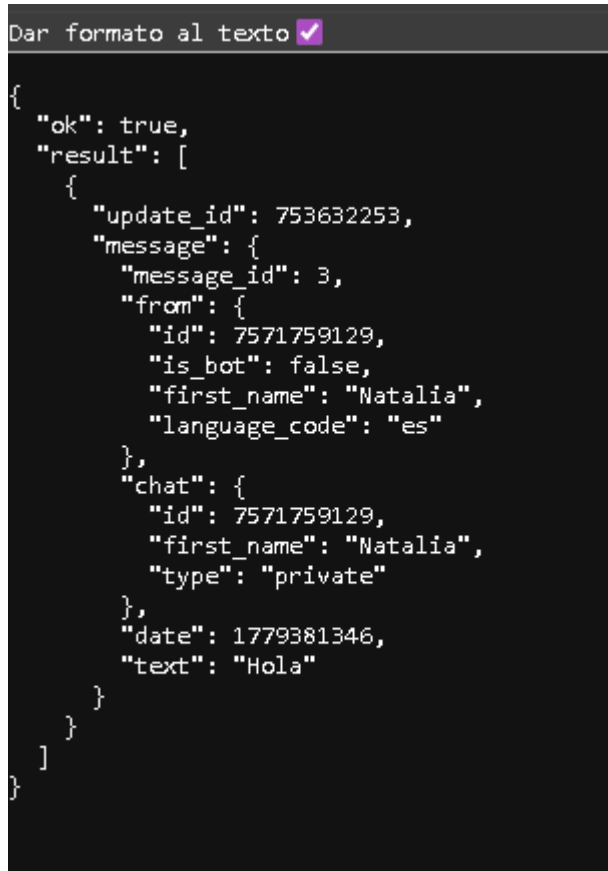
Mensaje



7. Abre el navegador y copia esta url con el token generado

<https://api.telegram.org/8822492274:AAFxUYQbl4CRE68r9q2xo27B7DVhXT7JRpw/getUpdates>

8. Se visualiza así



```
Dar formato al texto ✓  
{  
  "ok": true,  
  "result": [  
    {  
      "update_id": 753632253,  
      "message": {  
        "message_id": 3,  
        "from": {  
          "id": 7571759129,  
          "is_bot": false,  
          "first_name": "Natalia",  
          "language_code": "es"  
        },  
        "chat": {  
          "id": 7571759129,  
          "first_name": "Natalia",  
          "type": "private"  
        },  
        "date": 1779381346,  
        "text": "Hola"  
      }  
    }  
  ]  
}
```

9. Luego ve a jenkins y descarga

- Telegram bot

10. Instalalo reinicia jenkins

11. Configura el pipeline así

```
post {  
    success {  
        discordSend description: 'Pipeline ejecutado con  
EXITO',  
        link: env.BUILD_URL,  
        result: currentBuild.currentResult,  
        title: JOB_NAME,  
        webhookURL: '  

```

<https://discord.com/api/webhooks/15070441104044236>

82/vXsc0rTd3vMrFEe66nByZvxiF1fko2wEhVPAg1lFgnZgvg
k95zQkMUxjvBTwVXDwG200'

```
sh ""curl -s -X POST  
https://api.telegram.org/bot8822492274:AAFxUYQbl4CRE  
68r9q2xo27B7DVhXT7JRpw/sendMessage -d  
chat_id=7571759129 -d text=' Pipeline EXITOSO:  
'${JOB_NAME}' Build # '${BUILD_NUMBER}' finalizo  
correctamente.'""
```

```
}  
failure {  
    discordSend description: 'Pipeline FALLO',  
    link: env.BUILD_URL,  
    result: currentBuild.currentResult,  
    title: JOB_NAME,  
    webhookURL: 'TU_URL_DE_DISCORD_AQUI'
```

```
sh ""curl -s -X POST  
https://api.telegram.org/bot8822492274:AAFxUYQbl4CRE  
68r9q2xo27B7DVhXT7JRpw/sendMessage -d  
chat_id=7571759129 -d text=' Pipeline FALLIDO:  
'${JOB_NAME}' Build # '${BUILD_NUMBER}' presento  
errores.'""
```

```
}  
}
```

12. Click en save y luego Build now

Status

Changes

Construir ahora

Configurar

Borrar Pipeline

Stages

Rename

Pipeline Syntax

prueba-jenkins

Enlaces permanentes

- "Última ejecución (#15) hace 16 Seg"
- "Última ejecución estable (#15) hace 16 Seg"
- "Última ejecución correcta (#15) hace 16 Seg"
- "Última ejecución fallida (#9) hace 2 Hor 10 Min"
- "Última ejecución fallida (#9) hace 2 Hor 10 Min"
- "Last completed build (#15) hace 16 Seg"

Builds >		
Filter		
Today		
✓ #15	16:41	▼
✓ #14	16:09	▼
✓ #13	15:02	▼
✓ #12	14:50	▼
✓ #11	14:34	▼
✓ #10	14:33	▼
✗ #9	14:30	▼
✓ #8	14:28	▼
✓ #7	14:26	▼
✓ #6	14:22	▼
✓ #5	14:17	▼
✓ #4	14:14	▼
✓ #3	14:12	▼

```
[01:34mINFO][m] B[1m-----B[m
[01:34mINFO][m] B[1;32mBUILD SUCCESSB[m
[01:34mINFO][m] B[1m-----B[m
[01:34mINFO][m] Total time: 1.694 s
[01:34mINFO][m] Finished at: 2026-05-21T16:41:31Z
[01:34mINFO][m] B[1m-----B[m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] discordSend
Sending notification to Discord.
[Pipeline] sh
+ curl -s -X POST https://api.telegram.org/bot8822492274:AAFxUYQb14CRE68r9q2x027870VhXT73Rpw/sendMessage -d chat_id=7571759129 -d text=✅ Pipeline EXITOSO: prueba-jenkins Build #15
finalizo correctamente.
{"ok":true,"result":{"message_id":4,"from":{"id":8822492274,"is_bot":true,"first_name":"Jenkins Notifier","username":"jenkins_notifier_natilla_bot"},"chat":
{"id":7571759129,"first_name":"Natalia","type":"private"},"date":1779381694,"text":"\u2705 Pipeline EXITOSO: prueba-jenkins Build #15 finalizo correctamente."}}
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```